Network Traffic Analysis, Attack Discovery, and Classification Using FLNET2023

John Gilbert

Abstract

This report presents a complete solution for the CSCI 5840 Computer Networks II network traffic analysis assignment using the FLNET2023 dataset. The work first reconstructs an approximate topology for the ten observed data-collecting routers using traffic-volume, feature-similarity, IP-overlap, and graph-analysis evidence. It then performs unsupervised attack discovery on the unlabeled traffic, comparing partitional, density-based, and model-based clustering. Finally, after labels are available, it trains supervised intrusion-detection models, evaluates class-imbalance strategies, develops an advanced calibrated random-forest model, and studies router-level generalization. The strongest supervised model reached 0.954 accuracy, 0.880 macro precision, 0.943 macro recall, 0.893 macro F1, and 0.957 weighted F1 on the labeled evaluation corpus.

I. Introduction

FLNET2023 is a flow-based network intrusion dataset generated from an emulation of the GEANT-2012 topology. In the assignment setting, ten routers D1–D10 collect CICFlowMeter-style flow records, initially with labels removed. The report follows the requested three-part structure: Task 1 reconstructs router topology; Task 2 discovers and semantically maps attacks without labels; and Task 3 uses labels for supervised multi-class attack classification.

All task scripts write reproducible tables and figures under `outputs/`. The report uses those artifacts directly and keeps the same router identifiers, class names, and sampling choices as the code. The report uses the IEEEtran one-column LaTeX template as required.

II. Task 1: Network Topology Reconstruction

A. Question 1.1: Exploratory Data Analysis

For each router, I computed flow counts, raw flow-duration summary statistics, the five most frequent destination ports, and the total forward-to-backward packet ratio. Table I shows the required summary. D10 carries the largest flow volume with 1,390,856 records, D8 has the highest mean duration, and D5 has the strongest forward/backward packet asymmetry. These differences suggest that the routers do not observe interchangeable traffic roles: D10 and D5 behave more like high-volume or aggregation vantage points, while D3 observes mostly short, low-volume flows.

TABLE I
Q1.1(a) per-router exploratory summary. Duration values are reported in the raw CICFlowMeter units used by the dataset.

Router	Flows	Min	Max	Mean	Median	Std	F/B	Top-5 destination ports
D1	205215	0	128510312	12767890	77	25386691	2.244	5050, 5001, 5002, 5004, 5003
D2	495778	0	129813622	11540025	13106609	9927155	1.460	5000, 7054, 6051, 7400, 5353
D3	221390	0	122032241	6515526	23	20409582	1.887	7002, 7003, 6053, 5051, 7005
D4	457963	0	121983284	15169981	15096316	13491365	1.464	2048, 7004, 7100, 7600, 6065
D5	521364	0	122202516	1877577	3994	3512127	2.828	8080, 5002, 6004, 7005, 5353
D6	611942	0	22326336	9043107	10786316	7754329	1.460	5000, 7051, 7053, 5002, 7054
D7	566758	0	122077234	12235674	44	15723953	1.461	6060, 7051, 7002, 7054, 5052
D8	470708	0	120751444	15822451	84	17704330	1.453	7123, 7053, 5002, 6050, 7052
D9	474181	0	121984841	10975938	11646	15531059	2.348	5055, 6050, 7052, 6003, 5003
D10	1390856	0	122029684	2086078	309	6504551	1.736	7777, 6003, 6004, 6001, 6002

Traffic volume was computed as total forward plus backward bytes per flow. Figure 1 shows the overlaid distribution summary. D5 has the highest median per-flow volume at 59,358 bytes, while D1 has the heaviest upper tail with a 99th percentile of 496,303.52 bytes. D3 has the lowest median volume at 224 bytes. High medians and high upper quantiles are stronger signs of aggregation or transit roles; low medians with long tails look more edge-like, with occasional bulk transfers.

For feature variance, I used a one-way ANOVA-style ratio: between-router mean-square variation divided by within-router mean-square variation. Identifier and timestamp fields were excluded. Table II and Figure 2 show that packet-size, byte-rate, and packet-rate fields separate routers most strongly, which is consistent with the idea that collectors observe different application mixes and backbone roles.

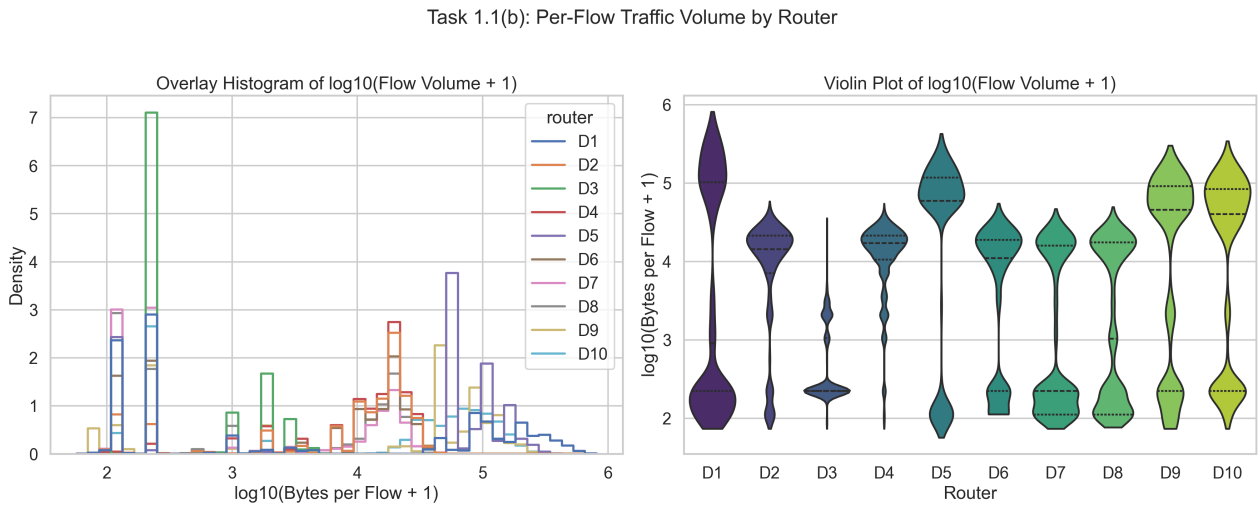

Fig. 1. Q1.1(b) per-router traffic-volume distributions using total bytes per flow.

TABLE II
Q1.1(C) TOP FEATURES BY BETWEEN-ROUTER TO WITHIN-ROUTER VARIANCE RATIO.

Feature	F-ratio	Interpretation
`bwd_pkts_b_avg`	644687.3	Flow-structure and traffic-composition separation
`pkt_len_var`	617539.2	Packet-size variability differs across routers
`fwd_byts_b_avg`	540440.2	Forward byte-rate profile differs across routers
`fwd_pkts_b_avg`	494239.7	Forward packet-rate profile differs across routers
`fwd_blk_rate_avg`	451936.3	Burst/block-rate structure differs across routers
`fwd_pkt_len_mean`	427311.3	Forward packet-size profile differs across routers
`fwd_seg_size_avg`	427311.3	Forward segment-size profile differs across routers
`fwd_pkt_len_std`	377781.3	Forward packet-size dispersion differs across routers
`bwd_pkt_len_mean`	355782.9	Backward packet-size profile differs across routers
`bwd_seg_size_avg`	355782.9	Backward segment-size profile differs across routers

TABLE III
Q1.2(B) GRAPH-CONSTRUCTION COMPARISON.

Method	Nodes	Edges	Avg. degree	Density
2-NN graph	10	12	2.4	0.267
Maximum spanning tree	10	9	1.8	0.200

B. Question 1.2: Router Similarity and Adjacency Estimation

For Q1.2(a), each router was represented by the mean of 79 numeric CICFlowMeter features after excluding `src_ip`, `dst_ip`, and `timestamp`. The per-router feature profiles were z-scored across routers, and cosine similarity was computed on the standardized profile vectors. Cosine similarity was chosen because it compares traffic-profile shape rather than raw volume, which matters because flow counts vary substantially across D1–D10. Because cosine similarity can be negative after z-scoring, Figure 3 uses a diverging scale centered at zero.

For Q1.2(b), I compared a 2-nearest-neighbor graph and a maximum-spanning-tree graph. Table III summarizes the resulting graph structures, and Figure 4 visualizes both. The kNN graph preserves local affinities but may add shortcuts that look less physically plausible. The maximum spanning tree is a better first-pass physical baseline because it guarantees a connected sparse backbone with exactly nine edges.

For Q1.2(c), I computed overlap of source/destination IP sets and Jaccard similarity of source-port, destination-port, and protocol tuples. The strongest overlap pairs were D10–D3, D6–D7, D8–D9, D6–D8, and D10–D7, as shown in Table IV. Figures 5–7 show the two overlap matrices and the refined graph separately so that the annotated values remain legible. The IP/tuple refinement retained seven feature-based MST edges, added D6–D7 and D8–D9, and removed D2–D8 and D5–D9. This means the feature-based and IP-based evidence reinforce many backbone edges, but IP overlap corrects pairs that appear globally similar without strong direct communication evidence.

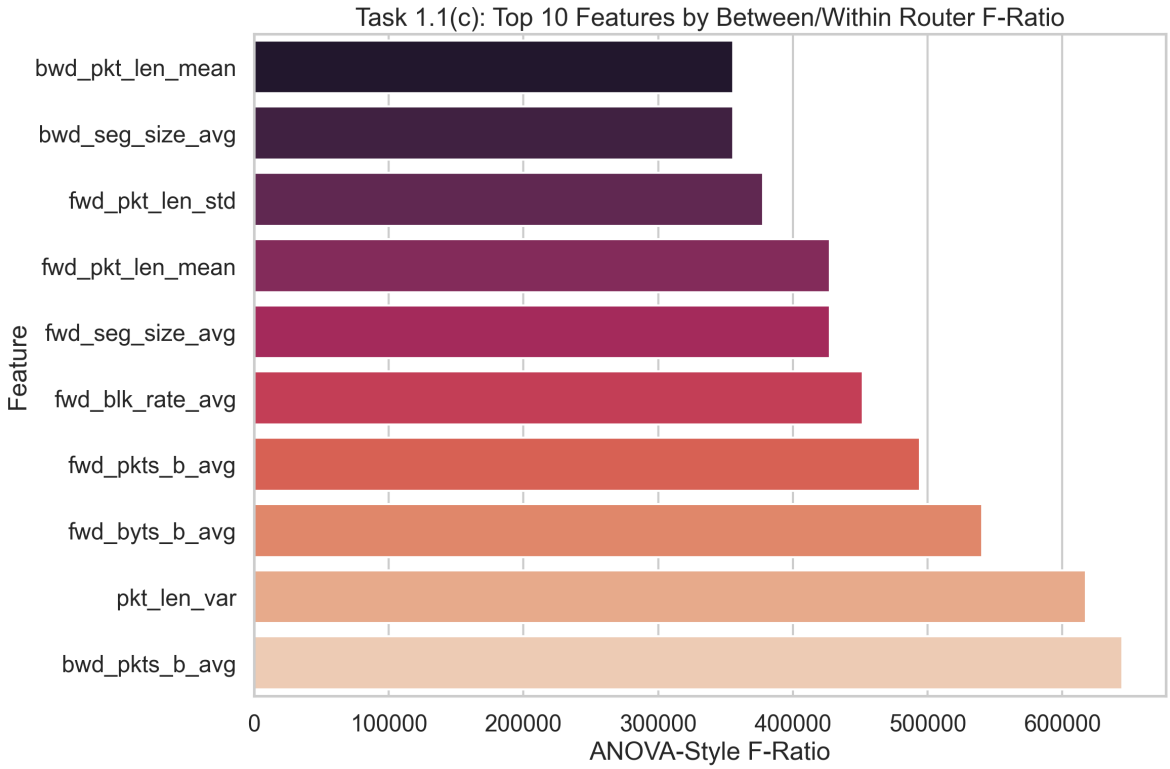

Fig. 2. Q1.1(c) top-10 features ranked by between-router to within-router variance ratio.

TABLE IV
Q1.2(C) STRONGEST IP AND PORT/PROTOCOL TUPLE OVERLAP PAIRS.

Router pair	Shared IPs	IP Jaccard	Tuple Jaccard
D10–D3	27	0.692	0.328
D6–D7	24	0.686	0.110
D8–D9	25	0.676	0.504
D6–D8	22	0.667	0.375
D10–D7	26	0.605	0.331

TABLE V
Q1.3(A) RECONSTRUCTED TOPOLOGY EDGES AND SELECTED EDGE WEIGHTS.

Source	Target	Tier	Composite	Direct evidence	Latency proxy
D7	D8	backbone	0.626	0.356	0.522
D6	D7	backbone	0.613	0.419	0.494
D8	D9	backbone	0.601	0.552	0.426
D2	D6	backbone	0.589	0.258	0.593
D5	D10	backbone	0.582	0.290	0.578
D3	D7	backbone	0.566	0.361	0.547
D3	D10	backbone	0.563	0.461	0.493
D2	D4	backbone	0.547	0.139	0.678
D1	D9	backbone	0.537	0.202	0.647
D9	D10	shortcut	0.463	0.309	0.622
D6	D8	redundancy	0.569	0.502	0.468

C. Question 1.3: Topology Reconstruction and Validation

The final reconstructed topology treats the ten collectors as a sparse projection of a larger backbone. I started with a maximum-spanning backbone and then added limited redundancy only when direct evidence was strong. Edge weights combine feature similarity, IP Jaccard, tuple Jaccard, dominant-port overlap, traffic-volume compatibility, duration compatibility, packet-balance compatibility, and a latency proxy. Table V lists the selected edges, and Figure 8 shows the reconstructed graph.

Each kept edge has either MST support, direct IP/tuple evidence, or a strong router-role compatibility signal. D7–D8 and D6–D7 combine high neighbor consistency with strong role evidence; D8–D9 and D6–D8 are strongly reinforced by IP and

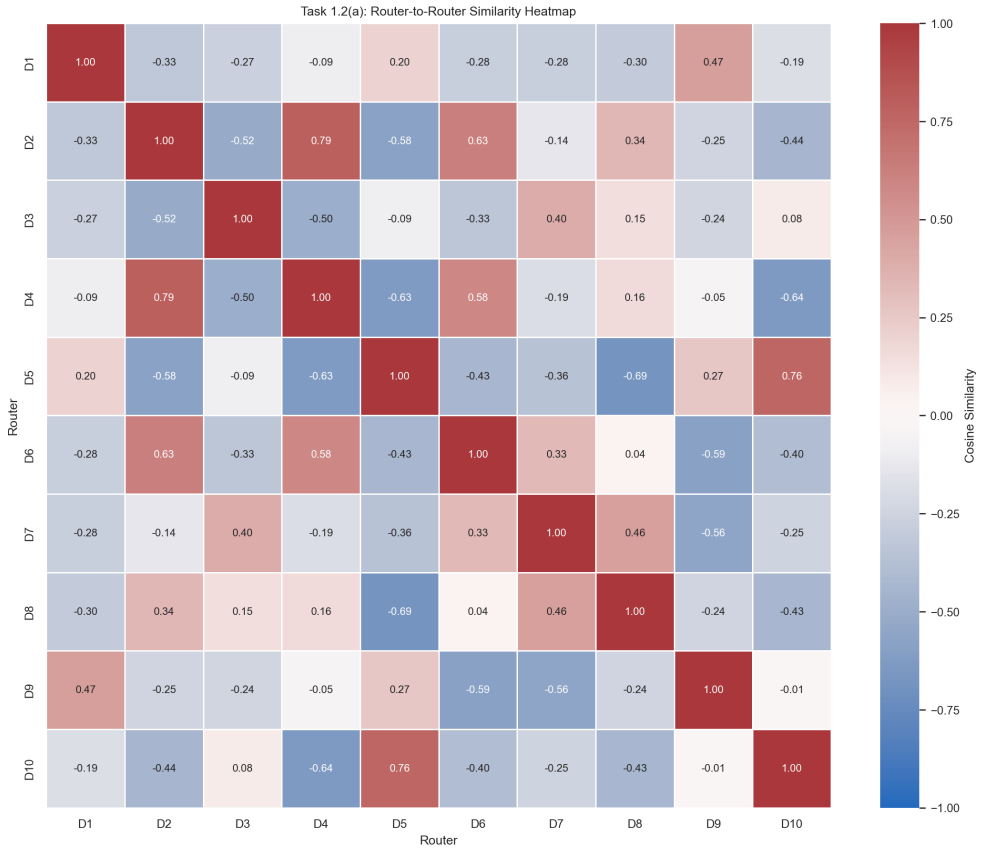

Fig. 3. Q1.2(a) cosine-similarity matrix over standardized per-router mean feature profiles. The diverging color scale is centered at zero so negative similarities are visually distinct from weak positive similarities.

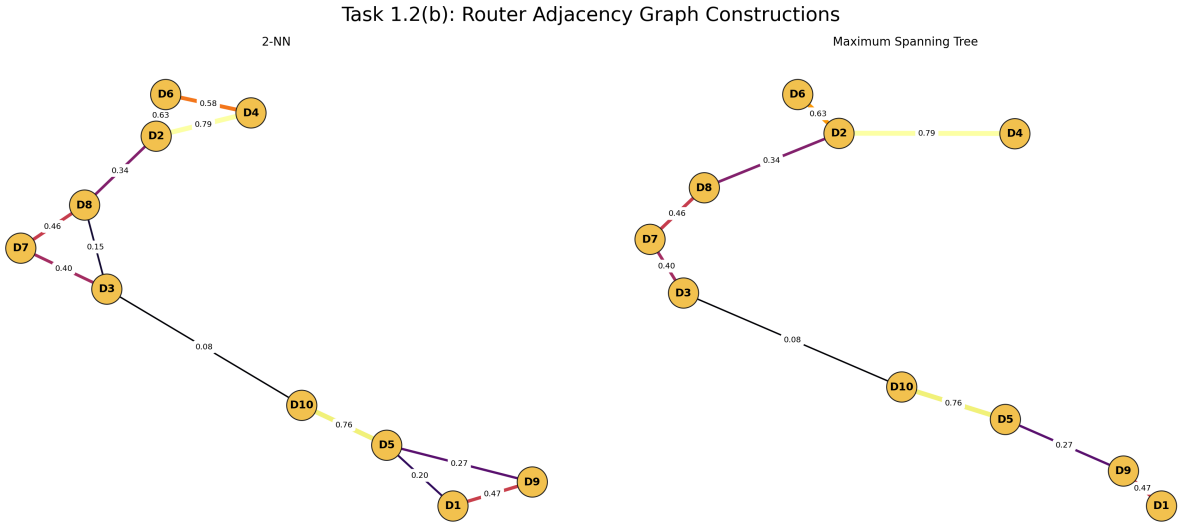

Fig. 4. Q1.2(b) force-directed adjacency visualizations for the kNN and maximum-spanning-tree approaches.

tuple overlap; D2–D6, D2–D4, and D5–D10 are primarily feature- and role-supported backbone links; D3–D7 and D3–D10 connect D3 to the central backbone with both feature and direct-overlap support; D1–D9 is D1's strongest sparse-backbone attachment; and D9–D10 is retained as a longer-path shortcut. Strong alternatives such as D7–D10, D5–D9, D2–D7, D8–D10, and D1–D10 were left out because the selected graph already connected those routers through better-supported paths or because direct overlap evidence was weaker.

The graph-theoretic properties are summarized in Table VI. The topology has 10 nodes, 11 edges, density 0.244, diameter 6

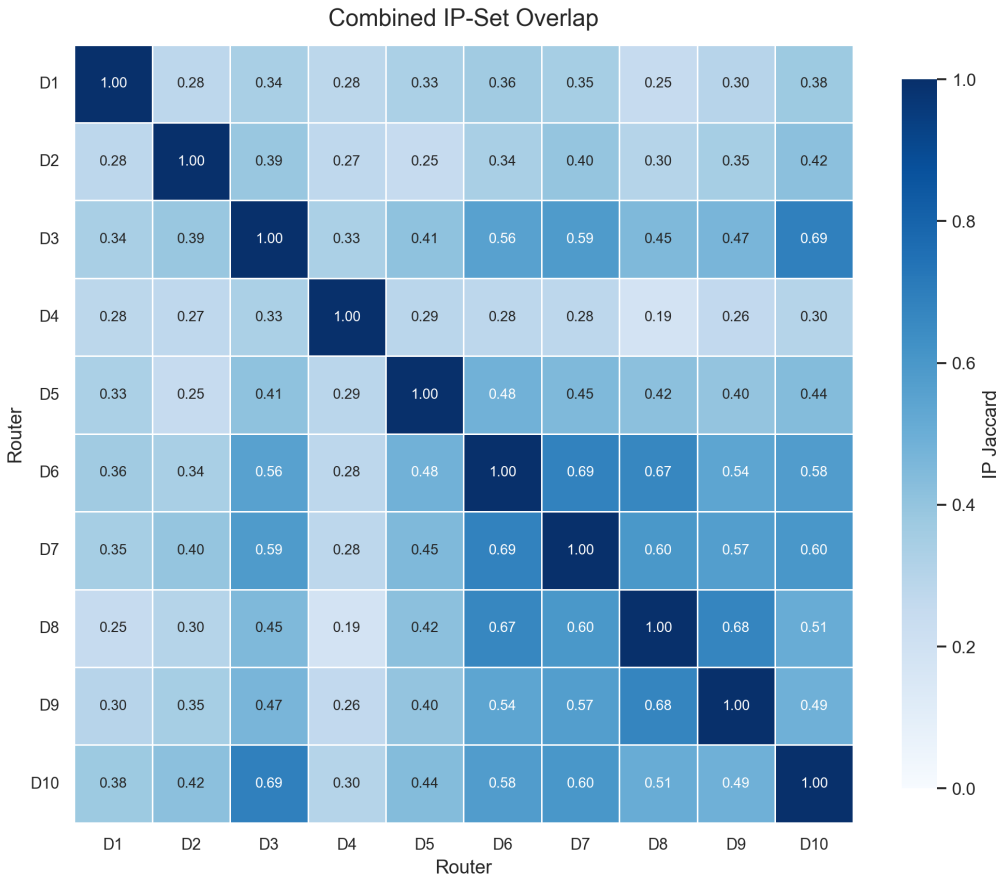

Fig. 5. Q1.2(c) IP-set Jaccard overlap matrix across routers.

TABLE VI
Q1.3(B) GRAPH-THEORETIC PROPERTIES FOR THE RECONSTRUCTED TOPOLOGY.

Property	Result
Nodes, edges, density	10 nodes, 11 edges, density 0.244
Degree distribution	degree 1: 3 routers; degree 2: 2 routers; degree 3: 5 routers
Diameter and average path length	6 hops; 2.511 hops average
Weighted path metrics	weighted diameter 3.366; weighted average path length 1.375
Highest betweenness	D9 = 0.444; D6 = 0.389; D8 = 0.389
Removal impact	Removing D9 leaves the graph disconnected, with two components and largest component size 8

hops, and average path length 2.511 hops. D9 has the highest betweenness centrality, followed by D6 and D8. Removing D9 fragments the graph into two connected components, so D9 behaves like the most critical bridge-like router in the reconstructed collector projection.

For Q1.3(c), I compared the reconstructed collector graph against public GEANT-2012 resources from Topology Zoo and TopoHub. The inferred graph correctly captured that GEANT behaves like one connected backbone with non-uniform transit importance. It diverged from reality in scale and exact geography: the inferred graph has only the ten monitored routers and 11 projected edges, while the public GEANT-2012 topology has 40 nodes and 61 links. The collector-only projection is also denser because unobserved intermediate routers are collapsed. Traceroute paths, RTT/geographic metadata, interface counters, or control-plane state would improve direct-adjacency inference.

The reconstruction is therefore not expected to be an exact physical topology. I attempted to infer links using only flow-observation evidence from D1–D10: feature-profile similarity, shared IP/port evidence, traffic-volume compatibility, packet-balance similarity, and sparse graph construction. That evidence can identify routers that see related traffic or occupy similar backbone roles, but it cannot reveal unlabeled routers that sit between the collectors. In the true topology, many D-router relationships pass through intermediate blue routers rather than through direct D-to-D links. For example, a strong inferred edge can mean that two collectors observe adjacent traffic domains, share a multi-hop path, or sit on the same service/attack

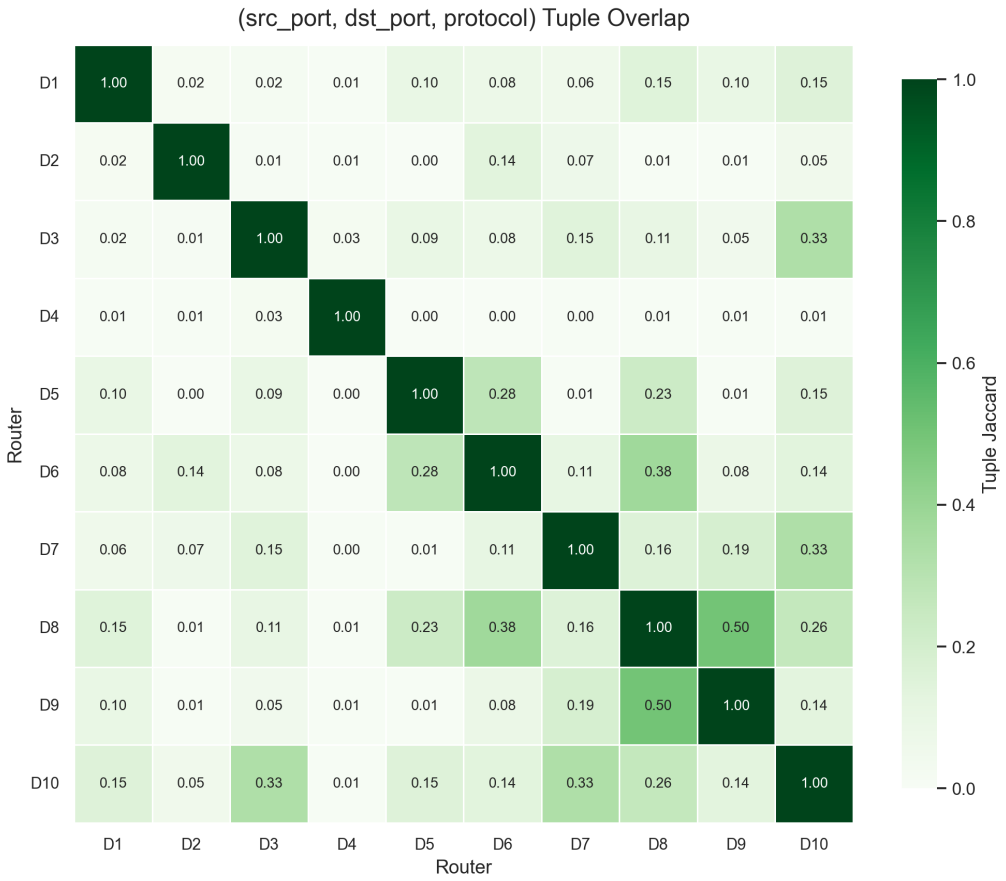

Fig. 6. Q1.2(c) Jaccard similarity of `(src_port, dst_port, protocol)` tuples across routers.

TABLE VII
Q1.3(C) COMPARISON WITH THE PUBLIC GEANT-2012 TOPOLOGY.

Metric	Inferred	GEANT-2012	Interpretation
Node count	10	40	The inferred graph covers only monitored collectors
Edge count	11	61	It is a sparse collector projection, not the full physical map
Density	0.244	0.078	Hidden intermediate routers are collapsed in the projection
Average degree	2.2	3.05	Broadly comparable, but semantics differ
Connected	Yes	Yes	Both views show a connected backbone
Highest centrality	D9	central European hubs	Both suggest non-uniform core transit importance

corridor. The result should be read as a ten-node collector-level projection of the network, not as the complete GEANT physical graph.

III. Task 2: Attack Discovery and Classification

A. Question 2.1: Preprocessing, Dimensionality Reduction, and Features

The preprocessing pipeline normalized router IDs, removed duplicates within each normalized router, converted missing and infinite values to missing values, imputed feature means, removed unusable or zero-variance features, and standardized the remaining numeric features. StandardScaler was used because PCA benefits from centered unit-variance inputs, while extreme network rates and counts can remain useful attack signals rather than pure outliers.

For Q2.1(b), streaming IncrementalPCA was fit on all 5,377,256 preprocessed rows. Eighteen principal components retained at least 95% of the variance, as shown in Figure 9. A UMAP embedding was then fit on a 100,000-row stratified sample preserving router distribution, using `n_neighbors=30` and `min_dist=0.1`. The same-router 15-nearest-neighbor purity was 0.6417 compared with a largest-router baseline share of 0.2541. Figure 10 shows weak-to-moderate router-driven grouping, but no clean partition by router alone.

For Q2.1(c), I engineered four attack-oriented features: directional byte imbalance, bytes per packet, burst/idle log ratio, and packet-size asymmetry. Table IX gives the networking intuition and computation for each. Their distributions across the final

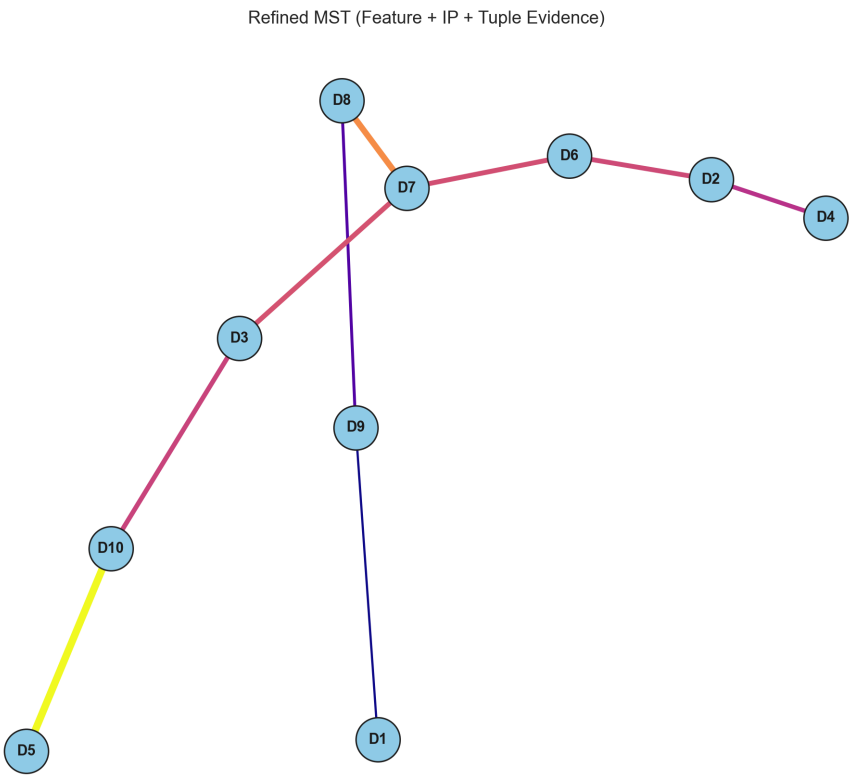

Fig. 7. Q1.2(c) refined maximum-spanning-tree graph after combining feature similarity, IP overlap, and tuple overlap.

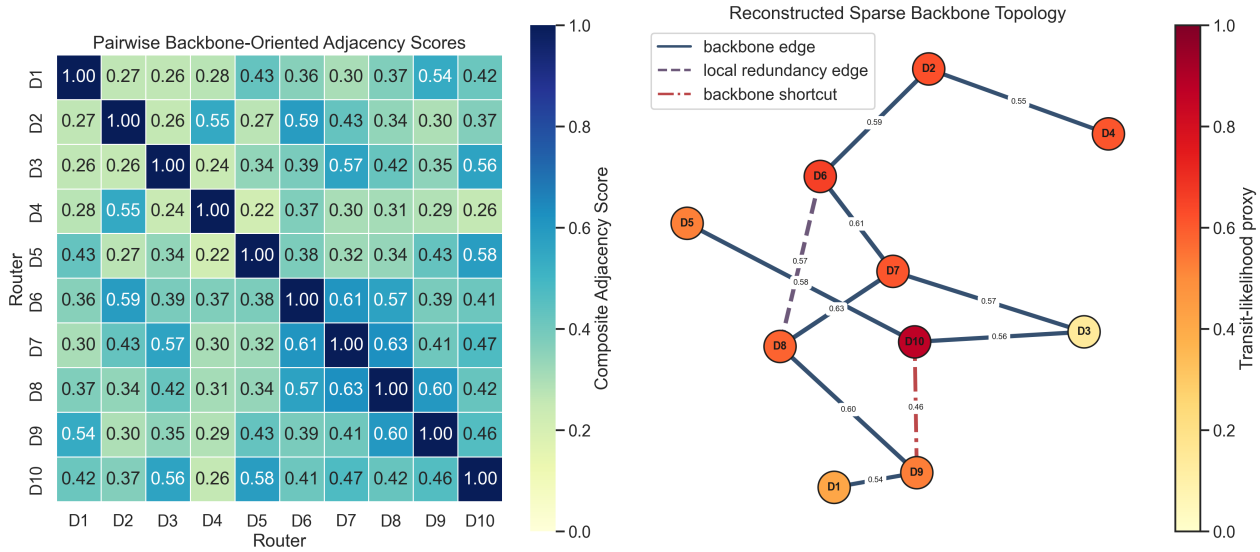

Fig. 8. Q1.3(a) reconstructed ten-router topology with weighted edge evidence.

HDBSCAN clusters are shown in Figure 11. The colors in that figure simply distinguish the HDBSCAN cluster groups shown on the x-axis; they do not encode an additional variable. Cluster -1 is HDBSCAN noise, while clusters 0–4 use the semantic labels assigned in Q2.2(c).

TABLE VIII
Q2.1(A) PREPROCESSING SUMMARY.

Item	Result
Rows before deduplication	5,416,155
Duplicate rows removed	38,899
Rows after deduplication	5,377,256
Features retained	67 numeric standardized features
Features removed	12 zero-variance or non-finite features: `ack_flag_cnt`, `bwd_psh_flags`, `bwd_urg_flags`, `cwe_flag_count`, `ece_flag_cnt`, `fwd_psh_flags`, `fwd_urg_flags`, `protocol`, `psh_flag_cnt`, `rst_flag_cnt`, `syn_flag_cnt`, `urg_flag_cnt`
Scaling	Z-score standardization with mean imputation for remaining numeric features

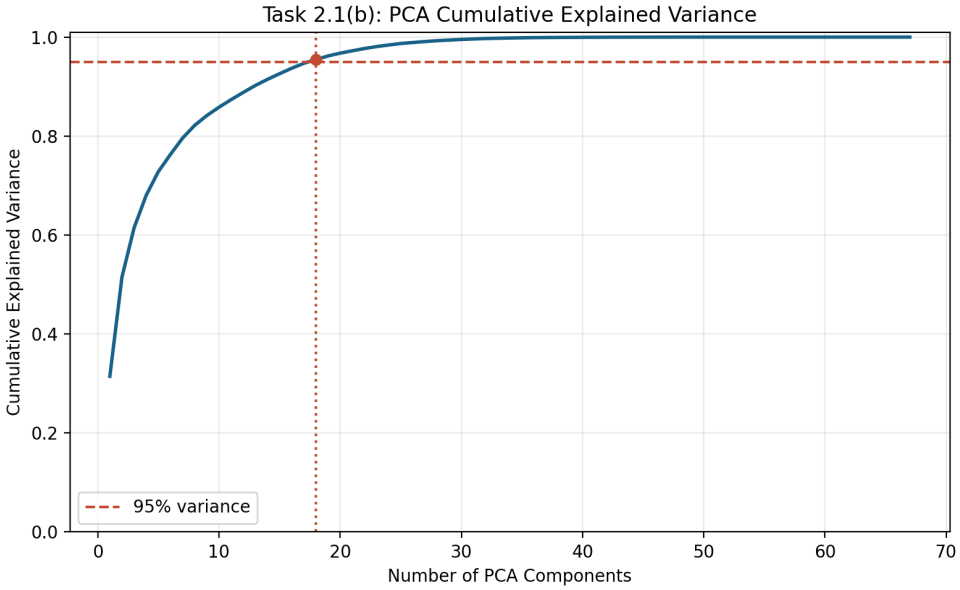

Fig. 9. Q2.1(b) PCA cumulative explained variance. Eighteen components retain at least 95% of variance.

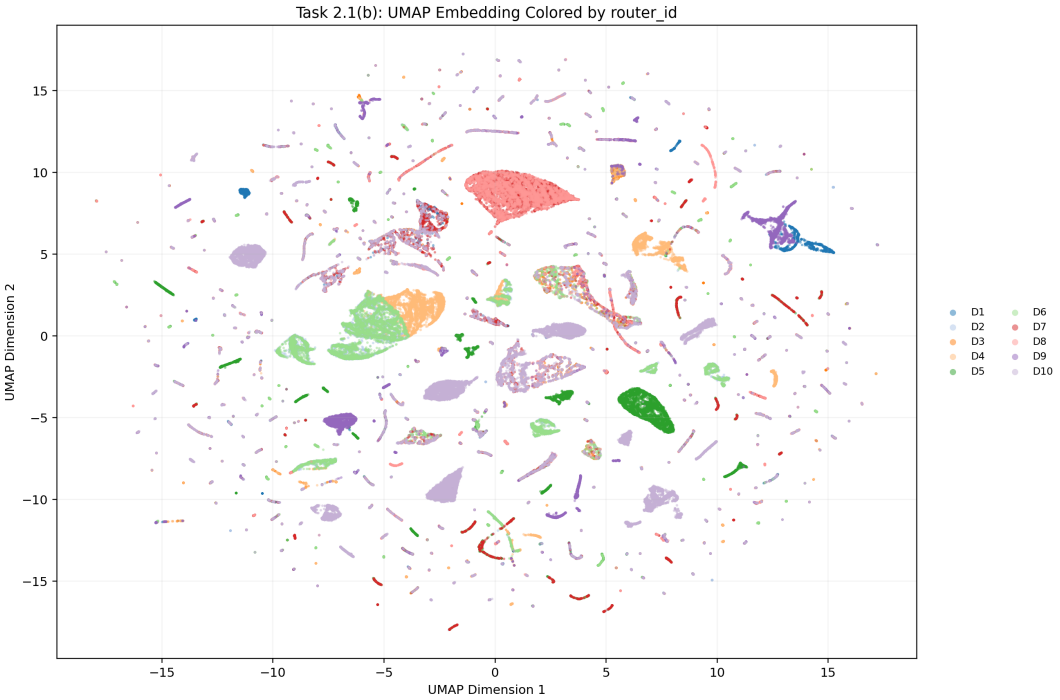

Fig. 10. Q2.1(b) UMAP embedding colored by router ID for the stratified 100,000-row sample.

TABLE IX
Q2.1(C) ENGINEERED FEATURES AND NETWORK INTUITION.

Feature	Computation	Intuition
`directional_byte_imbalance`	`(totlen_fwd_pkts - totlen_bwd_pkts) / (totlen_fwd_pkts + totlen_bwd_pkts + 1)`	Flooding and scanning can be highly one-sided
`bytes_per_packet`	`(totlen_fwd_pkts + totlen_bwd_pkts) / (tot_fwd_pkts + tot_bwd_pkts + 1)`	Attacks often have distinctive payload-size profiles
`burst_idle_log_ratio`	`log1p(max(active_mean, 0)) - log1p(max(idle_mean, 0))`	Scripted attacks often create concentrated bursts
`packet_size_asymmetry`	`abs(fwd_pkt_len_mean - bwd_pkt_len_mean) / (fwd_pkt_len_mean + bwd_pkt_len_mean + 1)`	Request-heavy or response-heavy attacks create directional size imbalance

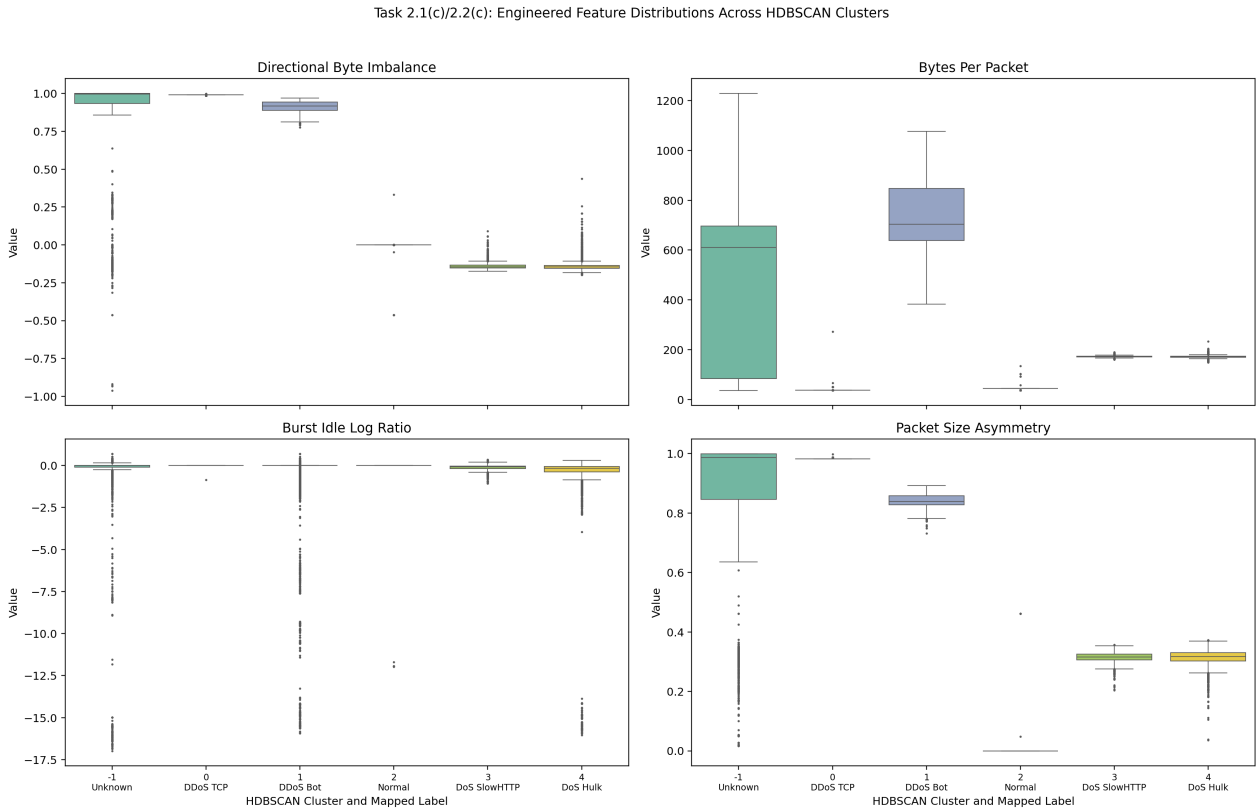

Fig. 11. Q2.1(c) engineered-feature distributions across the final discovered HDBSCAN clusters. Each boxplot shows the median, interquartile range, whiskers, and outliers for one engineered feature within one cluster; the x-axis labels give both cluster IDs and the semantic mappings used later in Q2.2(c).

B. Question 2.2: Unsupervised Attack Discovery

The clustering experiments reused the stratified 100,000-row UMAP sample and fit final clustering models on a stratified 20,000-row subset, with hyperparameter selection on an 8,000-row subset. This sampling choice follows the assignment's allowance for stratified random samples on computationally expensive clustering while giving the density-based model more support for stable density estimates. The three required clustering families were MiniBatchKMeans, HDBSCAN, and GaussianMixture. MiniBatchKMeans was selected by silhouette score over `k=8..14`; HDBSCAN was selected by silhouette score multiplied by non-noise coverage; and GaussianMixture was selected by lowest BIC over 8–14 components. Table X lists the selected configurations and cluster counts, and Figures 12, 13, and 14 show the embeddings.

TABLE X
Q2.2(a) SELECTED CLUSTERING ALGORITHMS AND HYPERPARAMETERS.

Algorithm	Family	Clusters	Selected configuration
MiniBatchKMeans	Partitional	8	`n_clusters=8`, `batch_size=4096`, `max_iter=200`, `n_init=10`
HDBSCAN	Density-based	5 plus noise	`min_cluster_size=1000`, `min_samples=25`, `cluster_selection_method=eom`; 12.29% noise
GaussianMixture	Model-based	11	`n_components=11`, diagonal covariance, `reg_covar=0.0001`, `max_iter=200`, `init_params=kmeans`

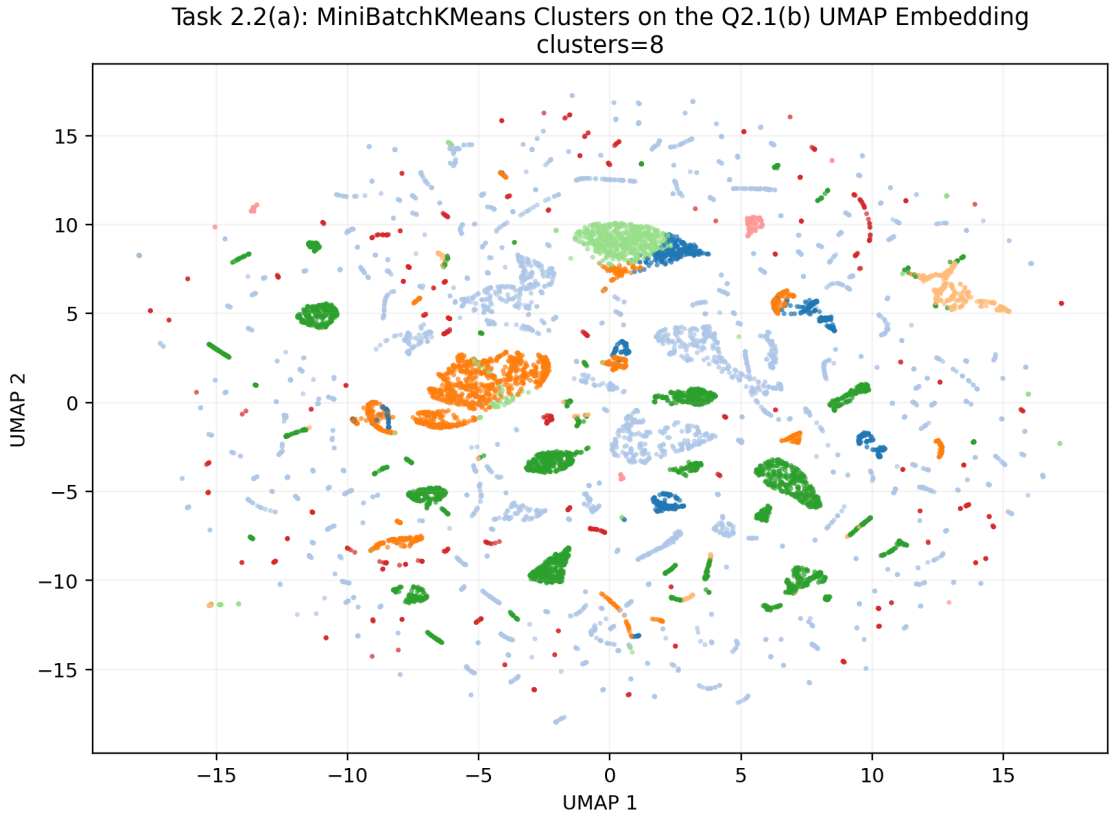

Fig. 12. Q2.2(a) MiniBatchKMeans cluster assignments projected onto the Q2.1(b) UMAP space.

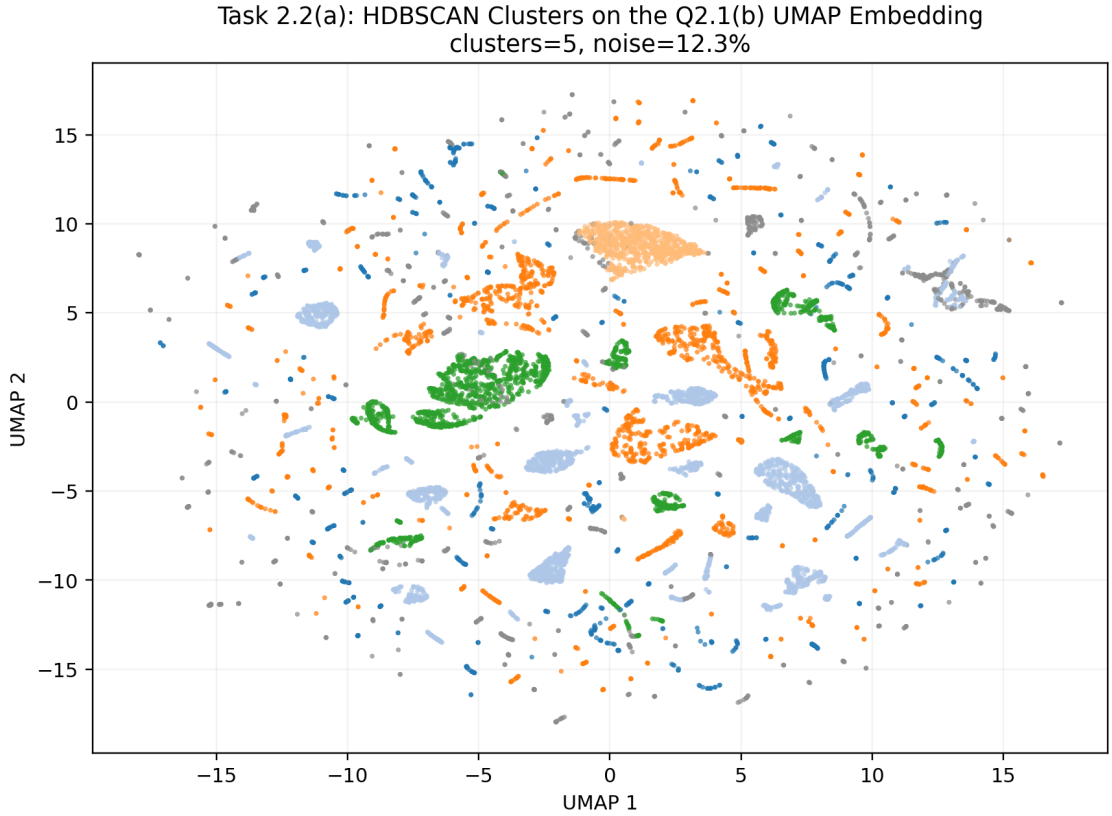

Fig. 13. Q2.2(a) HDBSCAN cluster assignments projected onto the Q2.1(b) UMAP space. Gray points are HDBSCAN noise.

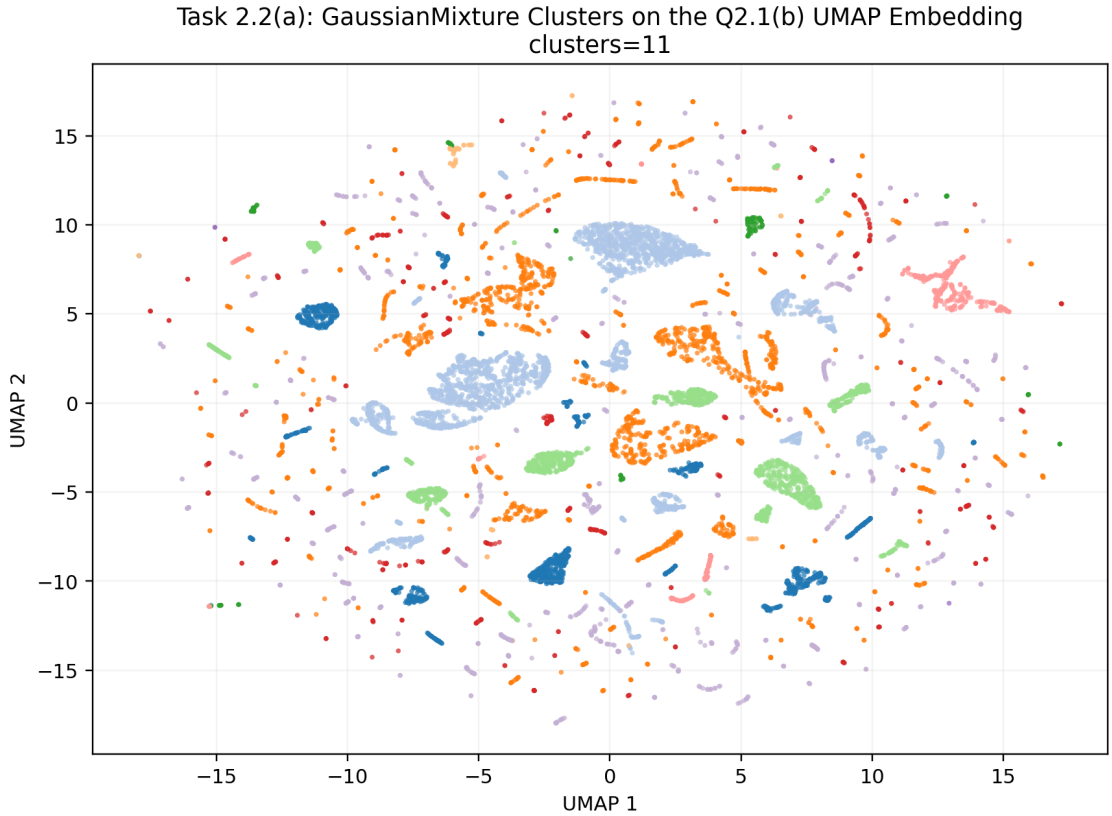

Fig. 14. Q2.2(a) GaussianMixture cluster assignments projected onto the Q2.1(b) UMAP space.

Internal validation used silhouette score, Davies-Bouldin Index, and Calinski-Harabasz Index. For HDBSCAN, noise points were excluded from metric computation because those metrics assume regular cluster labels. Table XI shows that HDBSCAN achieved the best overall rank, although GaussianMixture came closest to the nominal 11-class count.

TABLE XI
Q2.2(B) INTERNAL VALIDATION METRICS FOR CLUSTERING. HIGHER SILHOUETTE AND CALINSKI-HARABASZ ARE BETTER; LOWER DAVIES-BOULDIN IS BETTER.

Algorithm	Clusters	Noise	Noise %	Silhouette	DBI	CHI
HDBSCAN	5	2458	12.29	0.4599	0.9268	12571.89
GaussianMixture	11	0	0.00	0.4351	0.9922	5592.09
MiniBatchKMeans	8	0	0.00	0.4011	1.1477	4230.38

For Q2.2(c), HDBSCAN was selected for semantic mapping because it had the strongest internal validation profile and handles noise and imbalanced density better than methods that force every sample into a cluster. Table XII gives the confusion-style mapping to the known classes, and Table XIII gives the supporting cluster statistics. Cluster 0 maps to DDoS TCP because flows are tiny and one-directional; Cluster 1 maps to DDoS Bot because it contains large asymmetric flows concentrated on service ports; Cluster 2 maps to Normal because it contains short balanced exchanges with more port diversity; Cluster 3 maps to DoS SlowHTTP because it contains long-lived, low-rate flows; Cluster 4 maps to DoS Hulk because it has sustained request-burst behavior. Noise remains Unknown rather than being over-labeled. The protocol field was zero variance after preprocessing, so the dominant ports are the main service-level cue in the cluster statistics.

TABLE XII
Q2.2(C) HDBSCAN CLUSTER-TO-CLASS MAPPING.

Cluster	Predicted label	Rows	Share	Confidence
-1	Unknown	2458	12.29%	Low
0	DDoS TCP	1694	8.47%	High
1	DDoS Bot	5813	29.07%	Medium
2	Normal	4308	21.54%	Medium
3	DoS SlowHTTP	1570	7.85%	High
4	DoS Hulk	4157	20.79%	Medium

TABLE XIII
Q2.2(C) SUPPORTING HDBSCAN CLUSTER STATISTICS. DURATION VALUES ARE REPORTED IN THE RAW DATASET UNITS.

Cluster	Label	Mean duration	Mean packets	Mean bytes	Dominant destination ports
-1	Unknown	16.67M	57.85	33794	5055, 6050, 5050, 6051, 5000
0	DDoS TCP	146966	1.95	111	5003, 5002, 7005, 6004, 5055
1	DDoS Bot	3.39M	111.86	83561	7777, 8080, 5055, 5050
2	Normal	323	3.66	205	6003, 7002, 7005, 7054, 5051
3	DoS SlowHTTP	31.38M	101.80	17762	6060, 7123
4	DoS Hulk	13.76M	105.08	18321	5000, 2048, 6000

C. Question 2.3: Discovery Analysis

Discoverability was ranked using the selected HDBSCAN model, cluster mapping confidence, class distinctiveness, and assignment-provided imbalance facts. The easiest classes were those with extreme flow signatures, such as DDoS TCP and DoS SlowHTTP. The hardest were rare application-layer or stealthy behaviors such as Web Command Injection, Web SQL Injection, and Infiltration MITM, which are unlikely to form dense clusters from flow statistics alone.

TABLE XIV
Q2.3(A) INFERRED CLASS DISCOVERABILITY RANKING.

Class	Rank	Score	Reason
DDoS TCP	1	9.669	Minimal packets and one-way asymmetry
DoS SlowHTTP	2	9.157	Long-lived, very low-rate connections
DDoS Bot	3	7.581	Large asymmetric bot-driven floods
DoS Hulk	4	7.416	Burst-heavy HTTP flooding
Normal	5	5.631	Broad but still identifiable balanced traffic
DDoS Dyn	6	2.450	Overlaps with broader DDoS intensity signals
DDoS Stomp	7	2.450	Similar flooding profile to other DDoS variants
Web XSS	8	1.050	Application-layer abuse resembles normal web sessions
Infiltration MITM	9	-0.050	Stealthy behavior hidden in plausible exchanges
Web SQL Injection	10	-0.050	Rare and close to ordinary HTTP flow summaries
Web Command Injection	11	-0.550	Extremely rare; assignment states only 330 training instances

Severe imbalance shaped the cluster geometry. HDBSCAN found only five non-noise dense clusters and assigned 2,458 of 20,000 fitted rows to noise. GaussianMixture created 11 components including two microclusters with at most 100 rows. MiniBatchKMeans forced all rows into eight partitions with the largest cluster containing 32.11% of rows. These patterns indicate that rare attack types are treated as noise by HDBSCAN, fragmented by GaussianMixture, or absorbed into larger clusters by MiniBatchKMeans. Payload features, HTTP request metadata, TLS fingerprints, and host context would be especially helpful for the rare web and MITM classes.

IV. Task 3: Supervised Attack Classification

A. Question 3.1: Baseline Models

For Task 3, the training corpus contained 746,617 labeled rows from `data/test/`, and the evaluation corpus contained 5,416,155 labeled rows from `data/raw_labeled/`. The baseline pipeline used 79 numeric features and no categorical features. Table XV reports the required overall metrics for RandomForest, HistGradientBoosting, and MLP. HistGradientBoosting was used as the gradient-boosted tree baseline because the script falls back to scikit-learn's implementation when LightGBM/XGBoost native dependencies are unavailable. RandomForest was the strongest baseline with macro F1 0.8449 and weighted F1 0.9364.

TABLE XV
Q3.1(a) baseline model comparison.

Model	Time (s)	Acc.	Prec. macro	Rec. macro	F1 macro	F1 weighted	Key hyperparameters
RandomForest	60.809	0.9313	0.8510	0.9008	0.8449	0.9364	200 trees, unrestricted depth, min leaf 1, `n_jobs=-1`, seed 5840
HistGradientBoosting	41.540	0.5194	0.4868	0.4790	0.4203	0.6016	learning rate 0.05, 300 iterations, 63 leaf nodes, min leaf 20
MLP	156.728	0.7566	0.5731	0.7131	0.5568	0.7856	hidden layers 256 and 128, ReLU, Adam, batch size 4096, early stopping

TABLE XVI: Q3.1(a) per-class precision, recall, and F1 for the three baseline models.

Model	Class	Precision	Recall	F1	Support
RandomForest	DDoS-bot	0.426	0.995	0.597	87808
RandomForest	DDoS-dyn	0.717	0.911	0.802	322920
RandomForest	DDoS-stomp	1.000	0.692	0.818	389540
RandomForest	DDoS-tcp	0.952	0.883	0.916	930798
RandomForest	DoS-hulk	0.972	0.998	0.985	1644381
RandomForest	DoS-slowhttp	0.988	0.765	0.862	86226
RandomForest	Infiltration-mitm	0.367	0.999	0.537	28113
RandomForest	Normal	1.000	0.953	0.976	1922529
RandomForest	Web-command-injection	0.940	1.000	0.969	330
RandomForest	Web-sql-injection	1.000	1.000	1.000	441
RandomForest	Web-xss	1.000	0.712	0.832	3069
HistGradientBoosting	DDoS-bot	0.485	0.986	0.650	87808
HistGradientBoosting	DDoS-dyn	0.011	0.039	0.017	322920
HistGradientBoosting	DDoS-stomp	0.996	0.699	0.822	389540
HistGradientBoosting	DDoS-tcp	0.999	0.685	0.812	930798
HistGradientBoosting	DoS-hulk	0.642	0.268	0.378	1644381
HistGradientBoosting	DoS-slowhttp	0.362	0.047	0.084	86226
HistGradientBoosting	Infiltration-mitm	0.365	0.994	0.534	28113
HistGradientBoosting	Normal	0.860	0.692	0.767	1922529
HistGradientBoosting	Web-command-injection	0.001	0.130	0.002	330
HistGradientBoosting	Web-sql-injection	0.000	0.231	0.000	441
HistGradientBoosting	Web-xss	0.635	0.497	0.558	3069
MLP	DDoS-bot	0.316	0.900	0.468	87808
MLP	DDoS-dyn	0.149	0.521	0.232	322920
MLP	DDoS-stomp	0.992	0.951	0.971	389540
MLP	DDoS-tcp	0.998	0.999	0.998	930798
MLP	DoS-hulk	0.893	0.377	0.530	1644381
MLP	DoS-slowhttp	0.691	0.694	0.692	86226
MLP	Infiltration-mitm	0.452	1.000	0.622	28113
MLP	Normal	0.999	0.957	0.978	1922529
MLP	Web-command-injection	0.000	0.000	0.000	330
MLP	Web-sql-injection	0.009	0.948	0.018	441
MLP	Web-xss	0.804	0.497	0.615	3069

For Q3.1(b), RandomForest feature importances show that packet-shape and timing features dominate. The strongest features include `fwd_pkt_len_std`, `pkt_len_std`, `pkt_len_var`, `bwd_pkt_len_max`, `bwd_pkt_len_mean`, and `flow_duration`. This aligns with the Task 2 engineered features around packet-size asymmetry, bytes per packet, and burst/idle behavior. Figure 15 shows the top-20 importances.

For Q3.1(c), the best baseline RandomForest confusion matrix is shown in Figure 16. The most common confusions are DDoS Stomp predicted as DDoS Bot, DDoS TCP predicted as DDoS Dyn, Normal predicted as Infiltration MITM, Normal predicted as DDoS TCP, and DDoS Dyn predicted as DoS Hulk. These confusions are plausible because related DoS/DDoS families share flood-like timing and packet-shape features, while Infiltration MITM can preserve benign-looking client-server summaries.

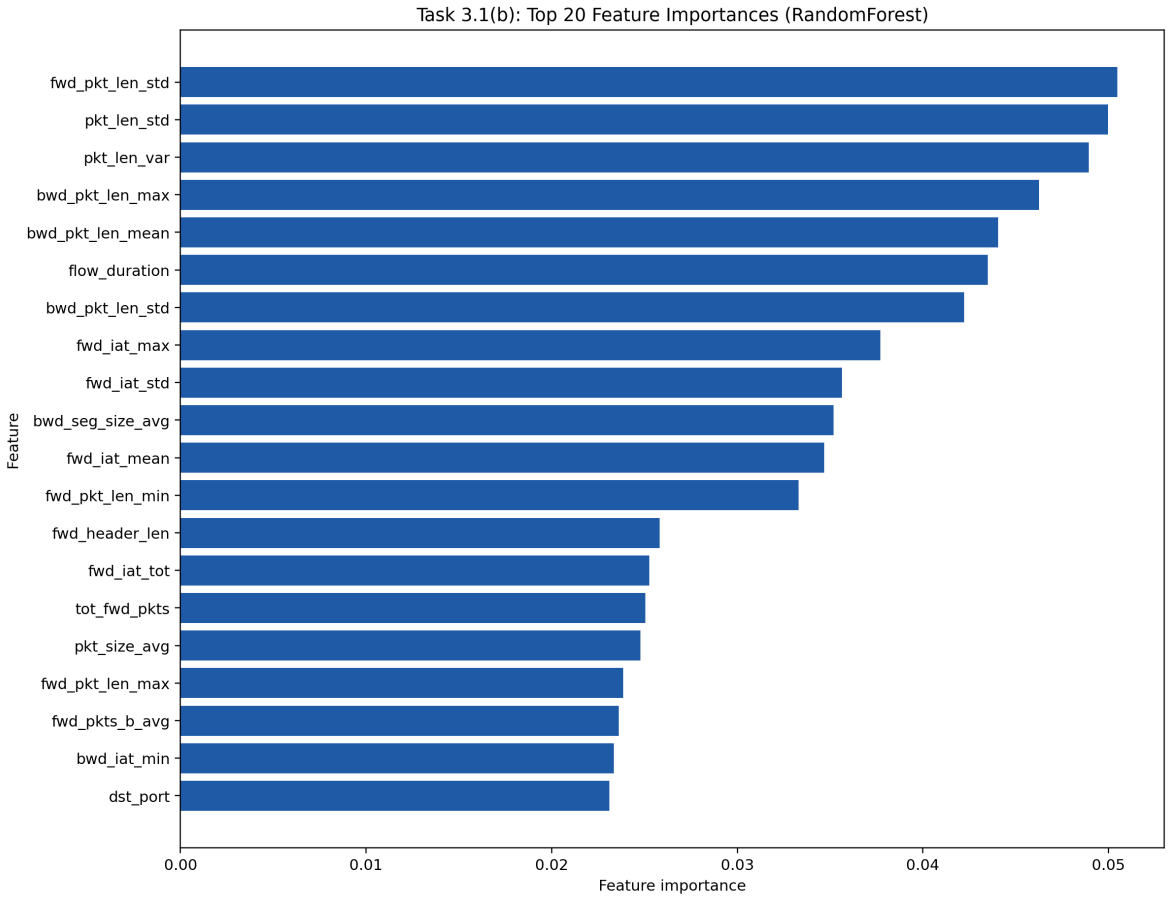

Fig. 15. Q3.1(b) top-20 RandomForest feature importances.

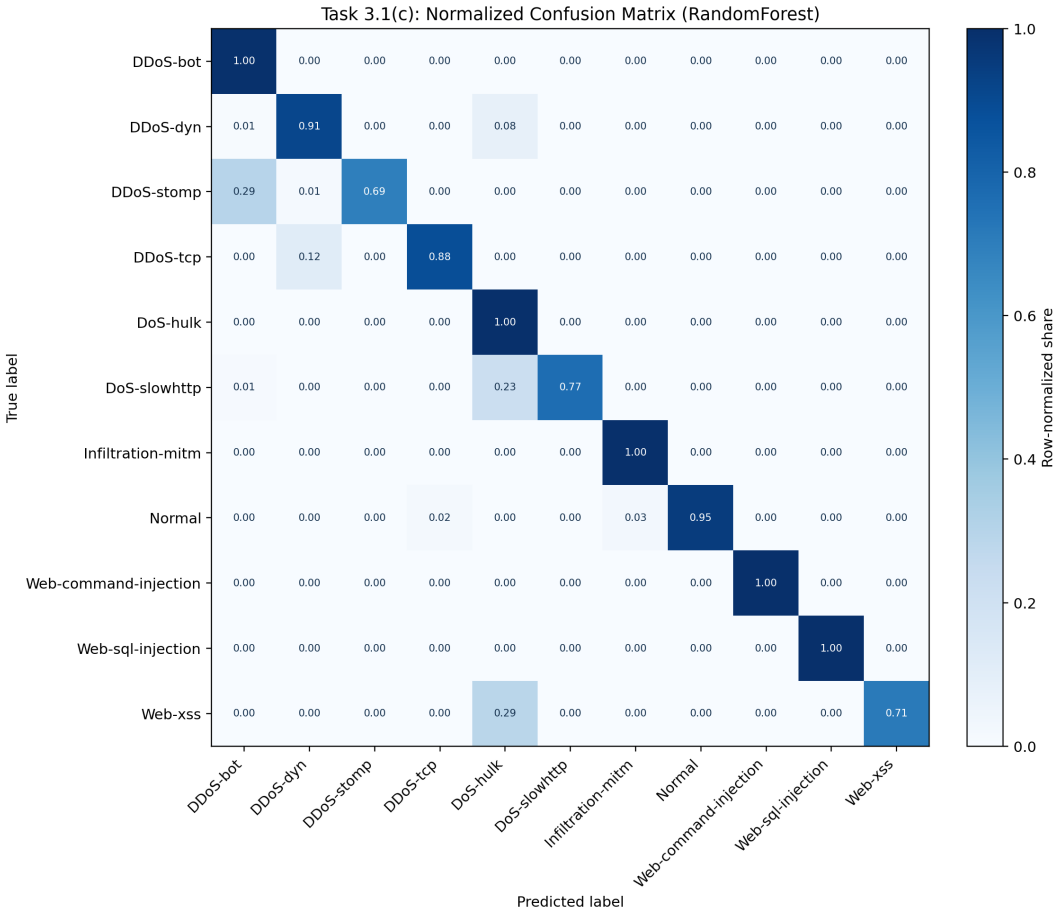

Fig. 16. Q3.1(c) confusion matrix for the best baseline RandomForest model.

B. Question 3.2: Handling Class Imbalance

I compared four RandomForest strategies: no balancing, selective SMOTE oversampling, class-weighted loss, and a hybrid SMOTE plus random undersampling strategy. Selective SMOTE oversampled classes below 25,000 training rows; the hybrid strategy capped the largest classes at 250,000 rows. Table XVII shows that no balancing had the best macro F1, while oversampling modestly helped Infiltration MITM but hurt the two rare web classes relative to the already strong baseline.

TABLE XVII
Q3.2(A) IMBALANCE-HANDLING STRATEGY COMPARISON.

Strategy	Time	Acc.	Prec. macro	Rec. macro	F1 macro	F1 weighted	Train rows
No Balancing	49.011	0.9313	0.8510	0.9008	0.8449	0.9364	746617
SMOTE Oversampling	61.272	0.9228	0.7924	0.8599	0.7839	0.9283	867548
Class Weighting	48.717	0.8862	0.6297	0.8629	0.6322	0.9135	746617
SMOTE + Undersampling	59.051	0.9206	0.7046	0.8563	0.6951	0.9268	815467

For Q3.2(b), Figure 17 shows rare-class F1 for Web Command Injection, Web SQL Injection, and Infiltration MITM. No balancing had the highest mean rare-class F1 at 0.8353 because the baseline already performed nearly perfectly on the two rare web classes. SMOTE improved Infiltration MITM from 0.5368 to 0.5950 but reduced Web Command Injection to 0.7341 and Web SQL Injection to 0.8786. Class weighting produced the largest precision collapse on rare web classes and also reduced majority-class F1.

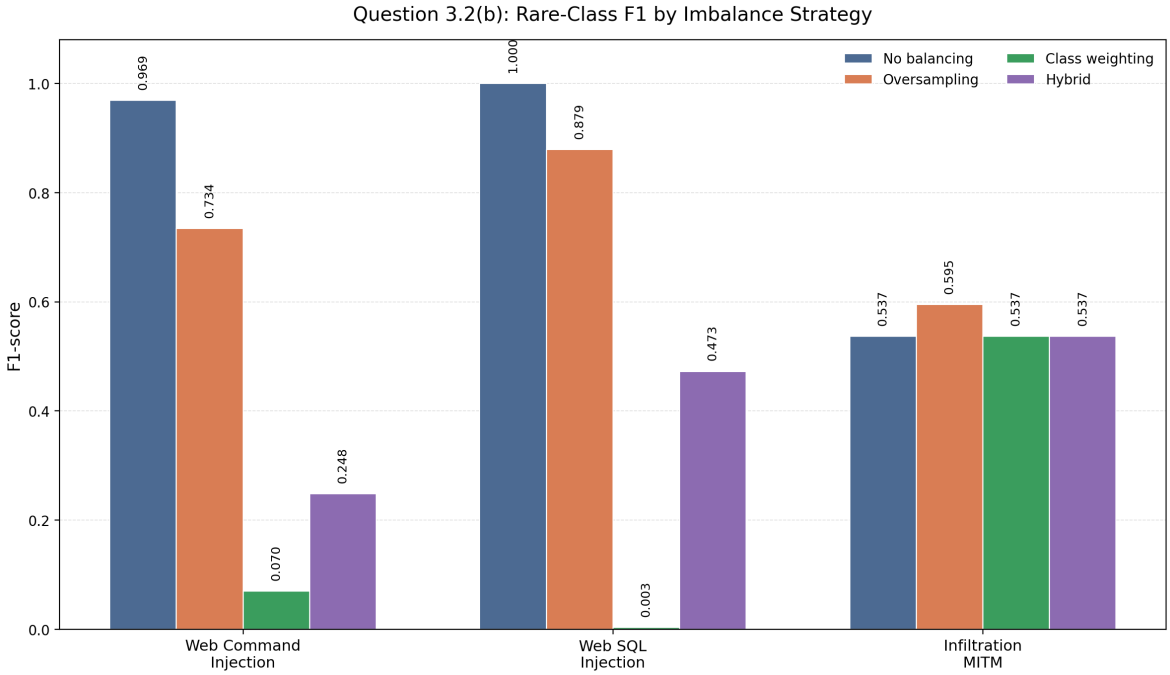

Fig. 17. Q3.2(b) rare-class F1 scores under the imbalance-handling strategies.

TABLE XVIII
Q3.2(B) RARE-CLASS AND MAJORITY-CLASS TRADEOFF SUMMARY.

Strategy	Mean rare F1	Weighted majority F1	Mean majority delta
No Balancing	0.8353	0.9385	0.0000
SMOTE Oversampling	0.7359	0.9301	-0.0466
Class Weighting	0.2035	0.9156	-0.0555
SMOTE + Undersampling	0.4192	0.9289	-0.0499

C. Question 3.3: Advanced Model

The advanced model family used a tuned RandomForest backbone with optional Task 2 engineered features and per-class score calibration. The hyperparameter search considered original versus engineered feature sets, `max_depth` in {None, 24},

`min_samples_leaf` in $\{1, 2\}$, and `class_weight` in {None, `balanced_subsample`}. The final selected configuration used original features only, 200 estimators, unrestricted depth, `min_samples_leaf=2`, no class weighting, and score multipliers for DDoS Bot and DDoS Dyn. Figure 18 summarizes the architecture.

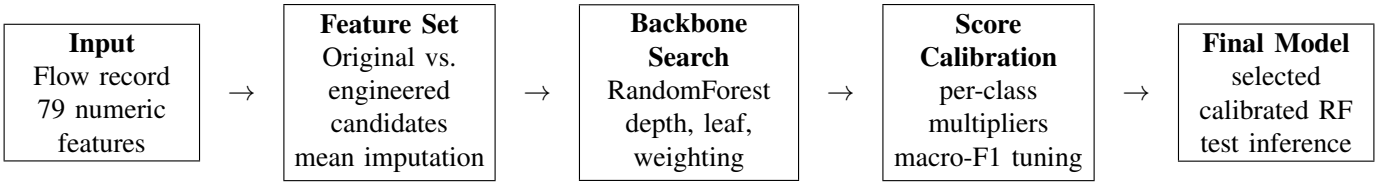

Fig. 18. Q3.3(a) advanced-model architecture. Candidate variants optionally added engineered features before the tuned RandomForest and calibration stage; the selected variant used original features only.

The development history matters for interpreting this result. The first advanced attempt, visible in the saved Task 3.3 postmortem and earlier git versions of the advanced-model script, used a more complex hybrid specialist design: a full RandomForest classifier, a binary web-traffic detector, and a kNN web-subtype specialist connected by an override rule. That model did not improve on the Q3.2 baseline. Macro F1 fell from 0.8449 to 0.5367 and accuracy fell from 0.9313 to 0.6576 because the web-specialist override fired on 1,496,705 evaluation rows, far more than the true rare web-attack subset. The override damaged both rare web precision and major non-web classes; for example, Web Command Injection precision fell to 0.0003 and DoS Hulk recall fell from 0.9985 to 0.2797. An ablation confirmed that the specialist was the main failure point: removing it raised macro F1 from 0.5367 to 0.6437, but still not enough to beat the simpler Q3.2 RandomForest. The final model therefore removed the specialist branch and kept only the tuned RandomForest with limited score calibration.

TABLE XIX
Q3.3(A) DEVELOPMENT HISTORY FOR THE ADVANCED MODEL.

Attempt	Accuracy	Macro F1	Interpretation
Best Q3.2: No Balancing RF	0.9313	0.8449	Baseline to beat.
Initial hybrid with web specialist	0.6576	0.5367	Too many web-specialist overrides; rare web precision and DoS Hulk recall collapsed.
Hybrid without specialist ablation	N/A	0.6437	Removing the specialist helped, but the model remained below the baseline.
Final calibrated RandomForest	0.9540	0.8930	Removed the specialist and used a tuned RF backbone with limited score calibration.

The final advanced model improved macro F1 from 0.8449 to 0.8930 relative to the best Q3.2 result. Table XX reports the required test-set metrics, and Table XXI gives per-class performance. The largest class-level gains relative to the baseline were on Web XSS, DDoS Bot, DDoS Stomp, and Infiltration MITM.

TABLE XX
Q3.3(A) FINAL ADVANCED-MODEL METRICS AND COMPARISON WITH BEST Q3.2 RESULT.

Model	Accuracy	Precision macro	Recall macro	F1 macro	F1 weighted
Best Q3.2: No Balancing	0.9313	0.8510	0.9008	0.8449	0.9364
Full Advanced	0.9540	0.8801	0.9432	0.8930	0.9571
Delta	0.0227	0.0291	0.0425	0.0481	0.0208

TABLE XXI
Q3.3(A) PER-CLASS METRICS FOR THE FINAL ADVANCED MODEL.

Class	Precision	Recall	F1	Support
DDoS-bot	0.549	0.977	0.703	87808
DDoS-dyn	0.756	0.915	0.828	322920
DDoS-stomp	0.999	0.821	0.901	389540
DDoS-tcp	0.999	0.901	0.948	930798
DoS-hulk	0.974	0.998	0.986	1644381
DoS-slowhttp	0.988	0.802	0.886	86226
Infiltration-mitm	0.424	0.999	0.595	28113
Normal	1.000	0.980	0.990	1922529
Web-command-injection	0.991	0.991	0.991	330
Web-sql-injection	1.000	0.993	0.997	441
Web-xss	1.000	0.997	0.999	3069

For Q3.3(b), the ablation study in Table XXII shows that score calibration was important, engineered features did not help on the full evaluation corpus, and class weighting hurt substantially. The largest macro-F1 drop came from enabling class weighting, which reduced macro F1 by 0.2522 relative to the selected model.

TABLE XXII
Q3.3(B) ABLATION STUDY.

Variant	Accuracy	Recall macro	F1 macro	Delta vs selected
Full Advanced	0.9540	0.9432	0.8930	0.0000
Add Engineered Features	0.9536	0.9016	0.8662	-0.0268
No Score Calibration	0.9310	0.9177	0.8534	-0.0396
Enable Class Weighting	0.9095	0.8705	0.6408	-0.2522

D. Question 3.4: Router-Level Analysis

For Q3.4(a), the selected advanced model was evaluated by router. Figure 19 shows router-by-class F1, and Table XXIII summarizes router-level performance. D6 was the easiest router with macro F1 over present classes of 0.9995, largely because its test mix is concentrated in DoS Hulk and Normal. D10 was hardest with macro F1 over present classes of 0.7447, driven by weak DoS SlowHTTP and Infiltration MITM performance under a DDoS TCP-heavy mix.

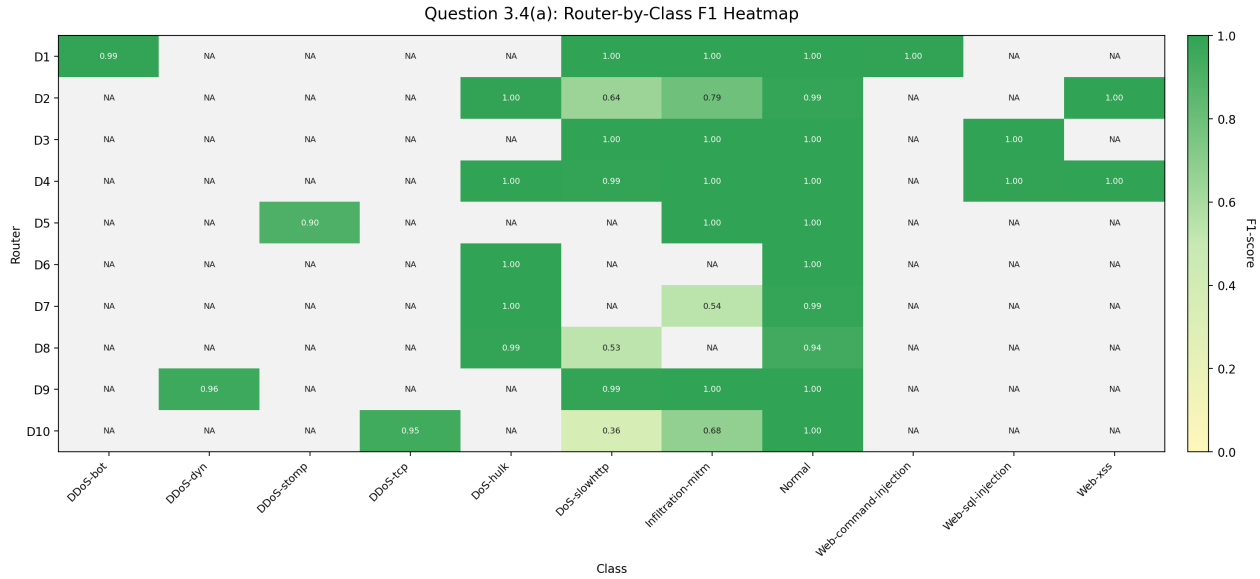

Fig. 19. Q3.4(a) per-router, per-class F1 heatmap for the selected advanced model.

TABLE XXIII
Q3.4(A) ROUTER-LEVEL PERFORMANCE SUMMARY. MACRO F1 IS COMPUTED OVER CLASSES PRESENT AT EACH ROUTER.

Router	Rows	Classes	Accuracy	Weighted F1	Macro F1	Dominant class	Dominant share	Worst class
D1	205215	5	0.990	0.995	0.997	Normal	0.507	DDoS-bot
D2	495778	5	0.987	0.988	0.883	DoS-hulk	0.793	DoS-slowhttp
D3	221390	4	1.000	1.000	0.999	Normal	0.882	Web-sql-injection
D4	457963	6	0.998	0.998	0.997	DoS-hulk	0.864	DoS-slowhttp
D5	521364	3	0.866	0.926	0.967	DDoS-stomp	0.747	DDoS-stomp
D6	611942	2	0.999	0.999	1.000	DoS-hulk	0.661	DoS-hulk
D7	566758	3	0.985	0.989	0.841	Normal	0.594	Infiltration-mitm
D8	470708	3	0.936	0.959	0.821	Normal	0.505	DoS-slowhttp
D9	474181	4	0.942	0.970	0.986	DDoS-dyn	0.681	DDoS-dyn
D10	1390856	4	0.926	0.959	0.745	DDoS-tcp	0.669	DoS-slowhttp

For Q3.4(b), I trained on routers D1–D7 and tested on unseen routers D8–D10. Table XXIV and Figure 20 show a severe drop: macro F1 fell from 0.8930 in the standard evaluation to 0.4870 cross-router. The main reason is class coverage rather than general failure on shared traffic: DDoS TCP and DDoS Dyn were absent from the training routers but made up 53.7% of the cross-router test rows. The model generalized well to shared classes on D8, but failed on unseen router-specific attack classes on D9 and D10. For distributed IDS deployment, this implies that training needs multi-router coverage, regular refreshes as routers come online, and monitoring for router-specific domain shift.

TABLE XXIV
Q3.4(B) STANDARD VERSUS CROSS-ROUTER GENERALIZATION METRICS.

Metric	Standard	Cross-router	Delta
Accuracy	0.9540	0.4623	-0.4917
Precision macro	0.8801	0.4859	-0.3942
Recall macro	0.9432	0.4894	-0.4538
F1 macro	0.8930	0.4870	-0.4060
F1 weighted	0.9571	0.4570	-0.5002

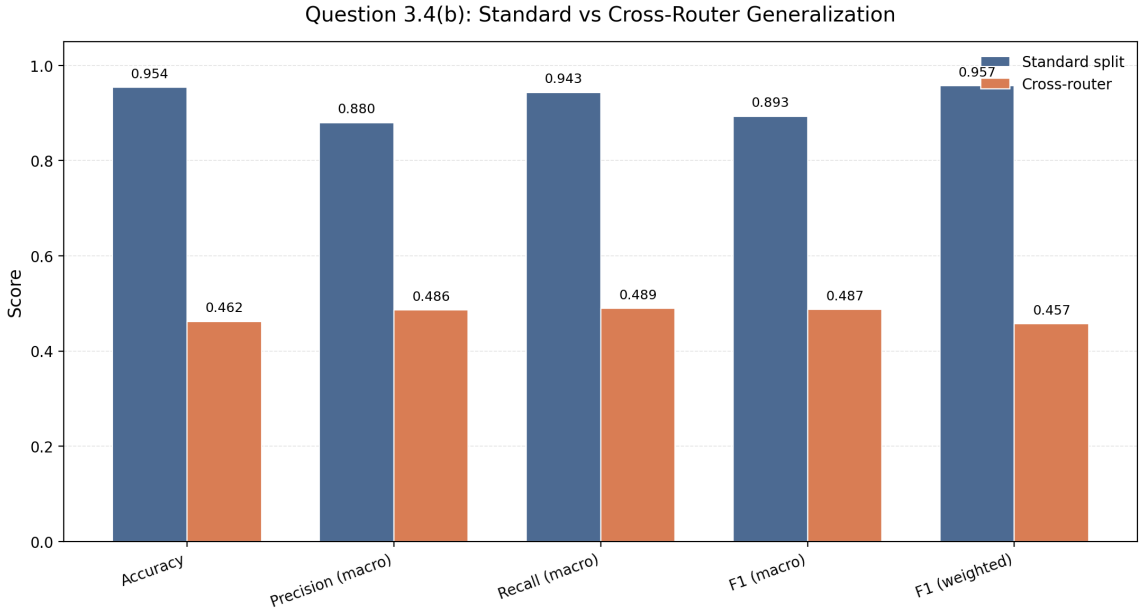

Fig. 20. Q3.4(b) standard evaluation versus cross-router generalization on D8–D10.

V. CONCLUSION

The topology analysis reconstructed a connected, sparse ten-router collector projection with D9, D6, and D8 acting as the most central routers. The reconstruction matched broad GEANT-2012 structure by recovering a connected backbone with non-uniform transit importance, but it could not recover hidden intermediate nodes or exact geography from flow data alone.

In the unlabeled discovery task, HDBSCAN was the strongest clustering model by internal validation, but it discovered only five dense non-noise clusters. The results show that flow-only unsupervised discovery is biased toward high-volume, low-entropy attack families such as DDoS TCP and DoS SlowHTTP, while rare web and MITM attacks need payload, endpoint, or session-context features.

In the supervised task, RandomForest was the strongest baseline and the final advanced calibrated RandomForest improved macro F1 to 0.8930. The router-level study showed high performance on routers whose classes were represented in training, but cross-router testing exposed poor generalization to attack classes absent from the training routers. A practical distributed IDS should therefore combine representative multi-router training, class-coverage checks, and continuous domain-shift monitoring.

References

[1] FML-Network reference repository, https://github.com/nsol-nmsu/FML-Network.
[2] Common Open Research Emulator, https://www.nrl.navy.mil/Our-Work/Areas-of-Research/Information-Technology/NCS/CORE/.
[3] CICFlowMeter project, https://github.com/ahlashkari/CICFlowMeter.
[4] Internet Topology Zoo dataset, https://topology-zoo.org/dataset.html.
[5] TopoHub topology catalog, https://www.topohub.org/.